Morphology

Challenges…

- How do we know that
  - Both *woorchunk* and woodchuncks have same original/root word?
    - May be easy: the plural just tacks as s on to end
  - But what about goose vs geese or fox vs foxes?
- **Two kinds of knowledge:**
  - **Orthographic rules**: can solve *woorchunk* vs. woodchuncks
  - **Morphological rules:** can distinguish goose vs geese

Orthographic/Spelling Rules

- General rules used when breaking a word into its stem and modifiers.
- Example:
  - Singular English words ending with –y, when pluralized, end with –ies.
  - Baby vs. babies

Morphological Rules

- Morphological rules are exceptions to the orthographic rules used when breaking a word into its stem and modifiers.
- Example:
  - Goose vs Geese is due to vowel change

Morphology: Definition

The study of words, how they are formed, and their relationship to other words in the same language.

Morphological Parsing

- **Parsing:** Take an input and produce some sort of linguistic structure
- Morphological parsing the process of determining the morphemes from which a given word is constructed.

Terminologies

	Tokens = N	Types = |V|
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
Google N-grams	1 trillion	13 million

Church and Gale (1990): $|V| > O(N^{½})$

- **Surface form:** Raw text present in a corpus
  - Example: going
- **Token:** a word present in running text (may be duplicated)
- **Type:** Unique word present in the running text
- **Vocabulary:** Set of types

they lay back on the San Francisco grass and looked at the stars and their

- 15 tokens (or 14)
- 13 types (or 12) (or 11?)

- **Stem**
- **Affix : prefix/suffix/infix/circumfix**

Infix

Tagalog: hingi (borrow) => h**um**ingi

Circumfix

German: Sagen (to say) =>ge sagt (said)

Terminologies

- A word can have more than one affix
  - Example: rewrites *(re-, write,-s),* unbelievably *(un-, believe, -able, -ly)*
- English doesn’t tend to stack more than 4 or 5 affixes
- Languages that tend to string affixes together like Turkish does are called **agglutinative** languages
  - Turkish can have words with 9 or 10 affixes

Terminologies

- **Inflection:** A word stem with a grammatical morpheme, usually resulting in a word of the same class as the original stem.
  - **Example:** Plural (-s) or past (-ed)

- **Derivation:** the combination of a word stem with a grammatical morpheme, usually resulting in a word of a different class
  - **Example:** computerize (verb) vs. computerization (noun)

Concatenative morphology is Easy!

- **Non-concatenative morphology**
  - Philipian language (Tagalog)
  - Um + hingi (request) = humingi (ask for)
- **Templatic morphology/root-and-pattern morphology**
  - In Hebrew, a verb is constructed from two components: a root (CCC) and a template (ordering of C and V to specify more semantic info)
  - *lmd* (learn/study) can be combined with active voice *CaCaC* template to produce *lamad* (he studied)
  - *CiCeC* => limed (he taught)
  - *CuCaC => lumad* (he was taught)

The Porter Stemmer (Porter, 1980)

- A simple rule-based algorithm for stemming
- An example of a HEURISTIC method
- Based on rules like:
  - ATIONAL -> ATE (e.g., *relational* -> *relate*)
- The algorithm consists of seven sets of rules, applied in order

The Porter Stemmer: definitions

- Definitions:
  - CONSONANT: a letter other than A, E, I, O, U, and Y preceded by consonant (e.g. SYZYGY)
  - VOWEL: any other letter
- With this definition, all words are of the form:
  $(C)(VC)^m(V)$
  C=string of one or more consonants (con+)
  V=string of one or more vowels
  m>=0
- E.g.,
  - Tr ou bl e s
  - C V C V C

The Porter Stemmer: rule format

- The rules are of the form:

  (condition) S1 -> S2

  Where S1 and S2 are suffixes

- Conditions:

m	The measure of the stem
*S	The stem ends with S
v	The stem contains a vowel
*d	The stem ends with a double consonant
*o	The stem ends in CVC (second C not W, X, or Y)

The Porter Stemmer: Step 1

- SSES -> SS
  - *caresses -> caress*
- IES -> I
  - *ponies -> poni*
  - *ties -> ti*
- SS -> SS
  - *caress -> caress*
- S -> ϵ
  - *cats -> cat*

The Porter Stemmer: Step 2a (past tense, progressive)

- (m>0) EED -> EE
  - Condition verified: *agreed -> agree*
  - Condition not verified: *feed -> feed*
- (*V*) ED -> ϵ
  - Condition verified: *plastered -> plaster*
  - Condition not verified: *bled -> bled*
- (*V*) ING -> ϵ
  - Condition verified: *motoring -> motor*
  - Condition not verified: *sing -> sing*

m	The measure of the stem
*S	The stem ends with S
v	The stem contains a vowel
*d	The stem ends with a double consonant
*o	The stem ends in CVC (second C not W, X, or Y)

The Porter Stemmer: Step 2b (cleanup)

- (These rules are ran if second or third rule in 2a apply)
- AT-> ATE
  - *conflat(ed) -> conflate*
- BL -> BLE
  - *Troubl(ing) -> trouble*
- (*d & ! (*L or *S or *Z)) -> single letter
  - Condition verified*: hopp(ing) -> hop, tann(ed) -> tan*
  - Condition not verified*: fall(ing) -> fall*
- (m=1 & *o) -> E
  - Condition verified*: fil(ing) -> file*
  - Condition not verified*: fail -> fail*

Why?

m	The measure of the stem
*S	The stem ends with S
v	The stem contains a vowel
*d	The stem ends with a double consonant
*o	The stem ends in CVC (second C not W, X, or Y)

(*V*) ED -> ϵ

Condition verified*: plastered -> plaster*

Condition not verified*: bled -> bled*

(*V*) ING -> ϵ

Condition verified*: motoring -> motor*

Condition not verified*: sing -> sing*

The Porter Stemmer: Steps 3 and 4

- Step 3: Y Elimination (*V*) Y -> I
  - Condition verified: *happy -> happi*
  - Condition not verified: *sky -> sky*
- Step 4: Derivational Morphology,
  - (m>0) ATIONAL -> ATE
    - *Relational -> relate*
  - (m>0) IZATION -> IZE
    - *generalization-> generalize*
  - (m>0) BILITI -> BLE
    - *sensibiliti -> sensible*

m	The measure of the stem
*S	The stem ends with S
v	The stem contains a vowel
*d	The stem ends with a double consonant
*o	The stem ends in CVC (second C not W, X, or Y)

The Porter Stemmer: Steps 5 and 6

- Step 5: Derivational Morphology, II
  - (m>0) ICATE -> IC
    - *triplicate -> triplic*
  - (m>0) FUL -> ϵ
    - *hopeful -> hope*
  - (m>0) NESS -> ϵ
    - *goodness -> good*
- Step 6: Derivational Morphology, III
  - (m>0) ANCE -> ϵ
    - *allowance-> allow*
  - (m>0) ENT -> ϵ
    - *dependent-> depend*
  - (m>0) ANT -> ϵ
    - *irritant -> irrit*
  - (m>0) IVE -> ϵ
    - *effective -> effect*

m	The measure of the stem
*S	The stem ends with S
v	The stem contains a vowel
*d	The stem ends with a double consonant
*o	The stem ends in CVC (second C not W, X, or Y)

The Porter Stemmer: Step 7 (cleanup)

- Step 7a
  - (m>1) E -> ϵ
    - *probate -> probat*
  - (m=1 & !*o) NESS -> ϵ
    - *goodness -> good*
- Step 7b
  - (m>1 & *d & *L) -> single letter
    - Condition verified*: controll -> control*
    - Condition not verified*: roll -> roll*

m	The measure of the stem
*S	The stem ends with S
v	The stem contains a vowel
*d	The stem ends with a double consonant
*o	The stem ends in CVC (second C not W, X, or Y)

Examples

- *computers*
  - Step 1, Rule 4: -> *computer*
  - Step 6, Rule 4: -> *compute*
- *singing*
  - Step 2a, Rule 3: -> *sing*
- *controlling*
  - Step 2a, Rule 3: -> *controll*
  - Step 7b : -> *control*
- *generalizations*
  - Step 1, Rule 4: -> *generalization*
  - Step 4, Rule 11: -> *generalize*
  - Step 6, last rule: -> *general*

Problems

- *elephants -> eleph*
  - Step 1, Rule 4: -> *elephant*
  - Step 6, Rule 7: -> *eleph*
- *Etc……*

Spelling Error: Minimum Edit Distance

How similar are two strings?

- Spell correction
  - The user typed “Appl”

    Which is closest?
    - App
    - Appeal
    - Apple

- Computational Biology
  - Align two sequences of nucleotides

    AGGCTATCACCTGACCTCCAGGCCGATGCCC
    TAGCTATCACGACCGCGGTCGATTTGCCCGAC
  - Resulting alignment:

    -AGG**CTATCAC**C**TGACCT**C**CA**G**G**C**CGA--TGCCC---**
    TAG-CTATCAC--GACCG**C--GG**T**CGA**TT**TGCCC**GAC

- Also for Machine Translation, Information Extraction, Speech Recognition

Edit Distance

- The minimum edit distance between two strings
- Is the minimum number of editing operations
  - Insertion **(I)**
  - Deletion **(D)**
  - Substitution **(S)**
- Need to transform one into the other

Minimum Edit Distance

- Two strings and their **alignment**: TRIAL vs ZEIL

```
T  R  I  A  L
|  |  |  |  |
Z  E  I  *  L
s  s     d
```

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
d s s     i s
```

- If each operation has cost of 1
  - Distance between these is 3
- If substitutions cost 2 (Levenshtein)
  - Distance between them is 5

Other uses of Edit Distance in NLP

- Evaluating Machine Translation and speech recognition

Spokesman confirms senior government adviser was shot

Spokesman said the senior adviser was shot dead

S I D I

- **Named Entity Extraction** and **Entity Coreference**
  - IBM Inc. announced today
  - IBM profits
  - US President Donald Trump announced yesterday
  - for United States President Donald Trump

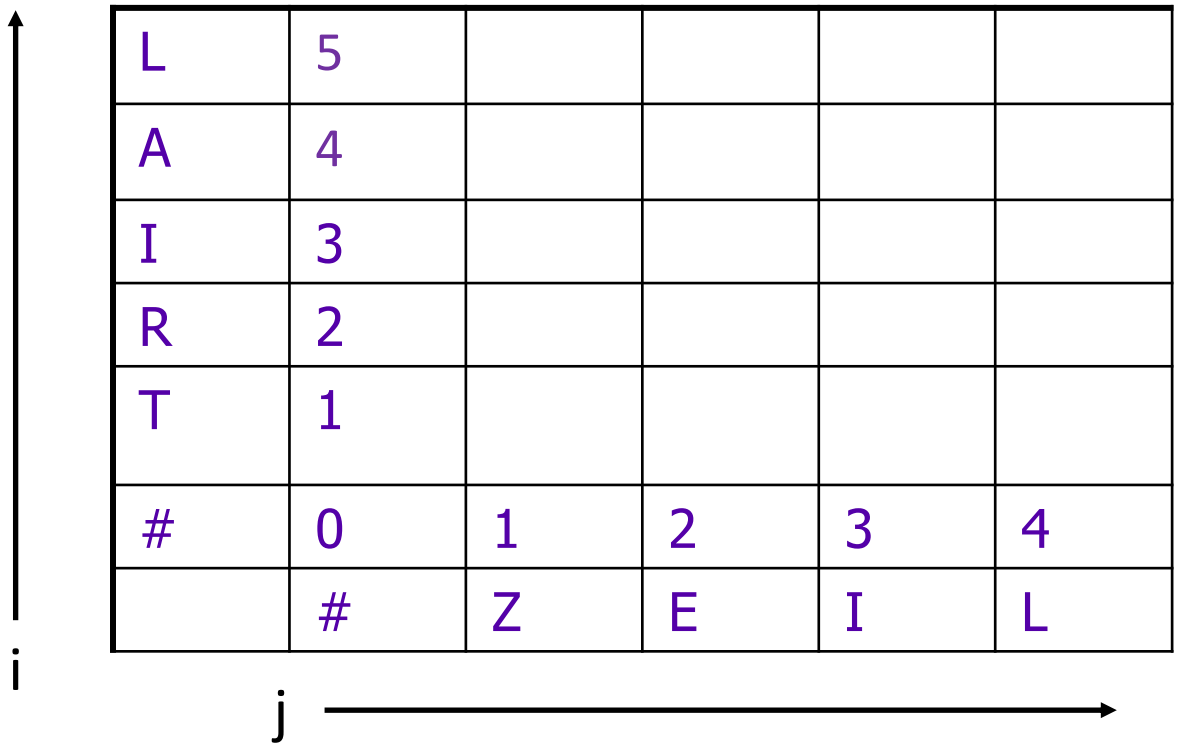
Minimum Edit as Search

- But the space of all edit sequences is huge!
  - We can't afford to navigate naïvely
  - Lots of distinct paths wind up at the same state.
    - We don't have to keep track of all of them

Defining Min Edit Distance

- For two strings
  - X of length n
  - Y of length m
- We define D(i,j)
  - the edit distance between X[1..i] and Y[1..j]
    - i.e., the first i characters of X and the first j characters of Y
- The edit distance between X and Y is thus D(n,m)

The Edit Distance Table



L	5				
A	4				
I	3				
R	2				
T	1				
#	0	1	2	3	4
	#	Z	E	I	L

The Edit Distance Table

i		j=0	1	2	3	4
5	L	5	6	7	6	**5**
4	A	4	5	6	5	6
3	I	3	4	5	4	5
2	R	2	3	4	5	6
1	T	1	2	3	4	5
0	#	0	1	2	3	4
		#	Z	E	I	L

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

The Edit Distance Table

(INTENSION vs EXECUTION)

N	9	8	9	10	11	12	11	10	9	**8**
O	8	7	8	9	10	11	10	9	8	9
I	7	6	7	8	9	10	9	8	9	10
T	6	5	6	7	8	9	8	9	10	11
N	5	4	5	6	7	8	9	10	11	10
E	4	3	4	5	6	7	8	9	10	9
T	3	4	5	6	7	8	7	8	9	8
N	2	3	4	5	6	7	8	7	8	7
I	1	2	3	4	5	6	7	6	7	8
#	**0**	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

Computing Alignments

- Edit distance isn't sufficient
  - We often need to **align** each character of the two strings to each other
- We do this by keeping a "backtrace"
- Every time we enter a cell, remember where we came from
- When we reach the end,
  - Trace back the path from the upper right corner to read off the alignment

Edit Distance with Backtrace

L	5	6	7	6	**5**
A	4	5	6	5	6
I	3	4	5	4	5
R	2	3	4	5	6
T	1	2	3	4	5
#	0	1	2	3	4
	#	Z	E	I	L

Substitute

No Change

Delete

T R I A L

| | | | | |

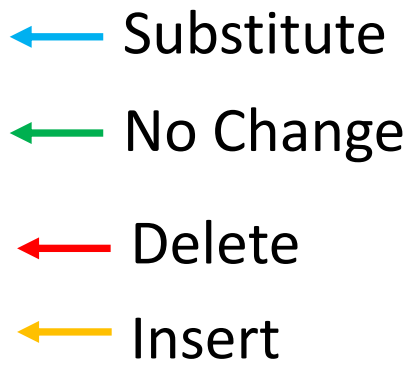
Z E I * L

s s d

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

Edit Distance with Backtrace (Another Path)

L	5	6	7	6	**5**
A	4	5	6	5	6
I	3	4	5	4	5
R	2	3	4	5	6
T	1	2	3	4	5
#	0	1	2	3	4
	#	Z	E	I	L



Cost is same, i.e., 5

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

MinEdit with Backtrace

n	9	↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↙←↓ 12	↓ 11	↓ 10	↓ 9	↙ **8**	
o	8	↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↓ 10	↓ 9	↙ **8**	← 9	
i	7	↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↓ 9	↙ **8**	← 9	← 10	
t	6	↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	↙ **8**	← 9	← 10	←↓ 11	
n	5	↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ **8**	↙←↓ 9	↙←↓ 10	↙←↓ 11	↙↓ 10	
e	4	↙ 3	← 4	↙← **5**	← **6**	← 7	←↓ 8	↙←↓ 9	↙←↓ 10	↓ 9	
t	3	↙←↓ 4	↙←↓ **5**	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙ 7	←↓ 8	↙←↓ 9	↓ 8	
n	2	↙←↓ **3**	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↓ 7	↙←↓ 8	↙ 7	
i	**1**	↙←↓ 2	↙←↓ 3	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙ 6	← 7	← 8	
#	**0**	1	2	3	4	5	6	7	8	9	
	#	**e**	**x**	**e**	**c**	**u**	**t**	**i**	**o**	**n**	

Adding Backtrace to Minimum Edit Distance

- Base conditions:

  $D(i,0) = i$ $\qquad$ $D(0,j) = j$

- Recurrence Relation:

  For each $i = 1 \ldots M$

  For each $j = 1 \ldots N$

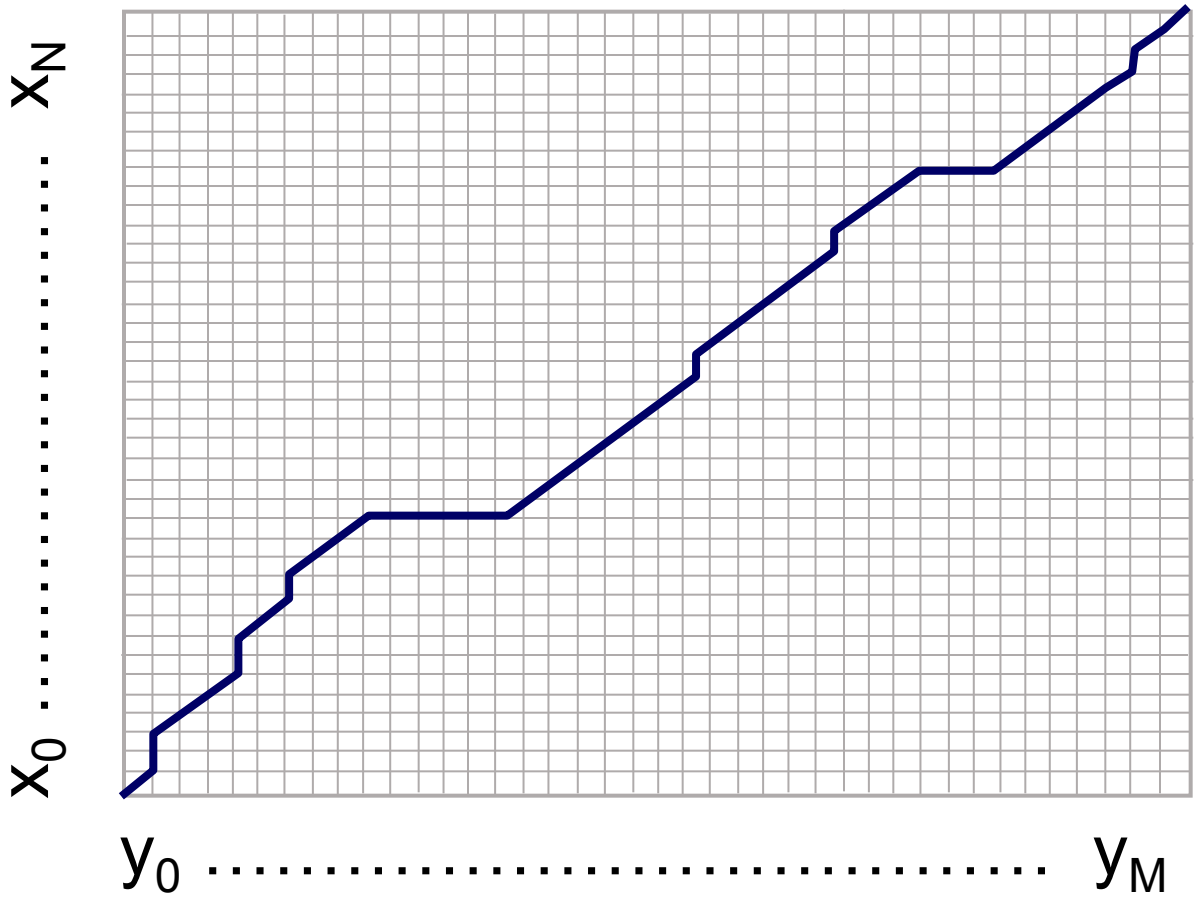
$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \text{deletion} \\ D(i,j-1) + 1 & \text{insertion} \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} & \text{substitution} \end{cases}$$

$$ptr(i,j) = \begin{cases} LEFT & \text{insertion} \\ DOWN & \text{deletion} \\ DIAG & \text{substitution} \end{cases}$$

Termination:

$D(N,M)$ is distance

The Distance Matrix



Every non-decreasing path

from (0,0) to (M, N)

corresponds to
an alignment
of the two sequences

An optimal alignment is composed of optimal subalignments

Performance

- Time: $O(nm)$
- Space: $O(nm)$
- Backtrace: $O(n+m)$